

Graph Retention Networks for Dynamic Graphs

Qian Chang
University of Auckland
Auckland, New Zealand
Central China Normal University
Wuhan, China
qcha783@aucklanduni.ac.nz

Runsong Jia
University of Technology Sydney
Sydney, Australia
runsong.jia@student.uts.edu.au

Xia Li*
Central China Normal University
Wuhan, China
lixia@ccnu.edu.cn

Jinqing Yang
Central China Normal University
Wuhan, China
yjy@mails.ccnu.edu.cn

Ciprian Doru Giurcaneanu
University of Auckland
Auckland, New Zealand
c.giurcaneanu@auckland.ac.nz

Xiufeng Cheng
Central China Normal University
Wuhan, China
xiufengcheng@ccnu.edu.cn

Guoping Hu
University of Auckland
Auckland, New Zealand
ghu206@aucklanduni.ac.nz

Abstract

In this paper, we propose **Graph Retention Networks** (GRNs) as a unified architecture for deep learning on dynamic graphs. The GRN extends the concept of retention into dynamic graph data as graph retention, equipping the model with three key computational paradigms: parallelizable training, low-cost $O(1)$ inference, and long-term chunkwise training. This architecture achieves an optimal balance between *efficiency*, *effectiveness*, and *scalability*. Extensive experiments on benchmark datasets demonstrate its strong performance in both edge-level prediction and node-level classification tasks with significantly reduced training latency, lower GPU memory overhead, and improved inference throughput by up to 86.7x compared to SOTA baselines. The proposed GRN architecture achieves competitive performance across diverse dynamic graph benchmarks, demonstrating its adaptability to a wide range of tasks.

CCS Concepts

• **Computing methodologies** → **Neural networks; Learning paradigms**; • **Theory of computation** → **Dynamic graph algorithms**.

Keywords

Dynamic Graphs, Graph Neural Networks, Temporal Learning, Scalability

ACM Reference Format:

Qian Chang, Xia Li, Xiufeng Cheng, Runsong Jia, Jinqing Yang, Guoping Hu, and Ciprian Doru Giurcaneanu. 2026. Graph Retention Networks for Dynamic Graphs. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792107>

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792107>

'26), April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792107>

1 Introduction

Dynamic graphs offer a flexible and powerful geometric framework for modeling complex real-world systems that evolve over time, such as ecosystems [18, 25], social networks [1, 16, 23], and traffic systems [13, 41, 44]. These structures capture the temporal dynamics of interactions between entities, enabling a deeper understanding of processes that unfold across both topological and temporal dimensions. Dynamic graphs exhibit progressive growth in scale and connectivity as interactions evolve, increasing both the modeling complexity and the computational demands of learning from such structures. Research has increasingly focused on modeling temporal dependencies and evolving relationships [5, 17, 43]. Unlike sequence-based retention models developed for natural language processing which assume linear and position-ordered token streams, dynamic graphs are characterized by asynchronous events and evolving topological dependencies that cannot be faithfully reduced to a single sequence [29, 31]. This structural discrepancy motivates a graph-native retentive formulation that jointly models temporal memory and structural propagation, rather than adapting retention mechanisms originally designed for NLP settings. However, three main challenges hinder the *efficiency*, *effectiveness*, and *scalability* of existing approaches as the prominence of dynamic systems grows.

One critical challenge leading to inefficiencies is the inability to reconcile training parallelism with low-cost inference [11, 20, 39, 48], particularly for dense, large-scale, and long-term dynamic graphs. While large-scale dynamic graphs demand parallel computation to process ever-growing and increasingly dense temporal interactions, most existing models fail to maintain this efficiency for inference [24, 34]. In practice, inference requires sequential or state-dependent retrieval of historical information, which breaks parallelism and introduces latency bottlenecks. This dual requirement imposes significant storage demands and computational overhead

[10, 49], particularly in real-time inference scenarios where timely access to both historical and current state information is essential.

Another pivotal barrier to the effectiveness of dynamic graph learning lies in establishing a principled balance between long-term and short-term temporal dependencies [22, 29, 34, 45–47]. Real-world dynamic systems often exhibit temporal heterogeneity, where certain relational patterns emerge gradually over extended durations, while others evolve rapidly within narrow temporal windows. Consequently, Temporal Graph Neural Networks (TGNNs) are expected to preserve long-term contextual information while remaining sensitive to recent fluctuations [15, 26, 42, 45]. Many existing methods lack adaptive mechanisms to regulate this temporal trade-off, resulting in representations that are either overly reactive to recent inputs or overly biased toward stale historical signals.

In terms of scalability, most existing models rely on auxiliary modules to manage temporal information flow. Typical approaches adopt separate components such as sampling or truncation strategies [37, 42, 45] and dedicated memory modules [16, 30, 38, 48] to approximate historical context. While such designs can reduce computational overhead or increase temporal coverage, they inevitably introduce architectural fragmentation. Sampling and truncation strategies risk discarding salient temporal signals due to local heuristics or fixed temporal windows, whereas memory-based approaches suffer from scalability bottlenecks [4]. The absence of a unified graph operator exacerbates these issues, limiting the scalability and broader applicability across diverse dynamic graph scenarios.

Contributions: This work proposes a unified and scalable architecture GRNs that addresses the three key challenges in dynamic graph learning. (i) We propose a graph retention mechanism to enable efficient parallel training and low-cost inference with reduced latency, lower memory overhead, and improved computational throughput. (ii) We introduce a multi-scale decay conjugated to a recurrent state that enables the model to balance long-term and short-term time dependencies for the representation improvement of various dynamic graph scenarios. (iii) By consolidating memory and aggregation into a unified architectural design without reliance on truncation or sampling-based heuristics, the GRN effectively eliminates modular inefficiencies and achieves strong scalability, generalizability, and architectural coherence across both continuous-time and discrete-time dynamic graphs.

2 Related Work

Early methods for learning on dynamic graphs typically utilized separate modules, with graph neural networks capturing structural information and recurrent neural networks handling temporal dependencies [16, 21, 26, 29, 42]. While this modular approach was intuitive, it faced significant scalability challenges [3, 7, 19], particularly when applied to large and densely connected graphs with extensive temporal information. Maintaining both structural and temporal embeddings significantly increased computational complexity [9, 51], severely impacting training efficiency.

To enhance the scalability of TGNNs, researchers have adopted techniques such as neighbor truncation [21, 45] and selective sampling [12, 29, 32, 46]. These approaches allow models to allocate computational resources more effectively by focusing on the most relevant temporal interactions. Such techniques are particularly

advantageous for real-time reasoning, where rapid inference and the ability to model both short-term and long-term dependencies are essential. Recent advances in TGNNs have introduced strategies to improve training efficiency on large-scale dynamic graphs. These include advanced data batching techniques, optimized memory management pipelines, and GPU utilization strategies [35, 45]. By reducing data access costs [11], minimizing redundant data handling [12], and streamlining data transfers [6, 24, 28], these innovations significantly boost throughput and lower computational burdens, enabling models to handle high-dimensional datasets effectively while maintaining their performance.

Unified architectures have emerged as a promising alternative, integrating topological and temporal information into a single framework. These models leverage unique graph operators [48, 50], truncation strategies [45], and temporal encoding functions [8, 15] to maintain temporal context without the need for post-training memory updates. Attention-based methods and causal masking further enhance these architectures by selectively aggregating information, enabling efficient capture of long-term dependencies in dynamic graph data while avoiding computational inefficiencies.

3 Preliminaries

3.1 Dynamic Graphs

A graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is an irregular data structure composed of a set of nodes (also referred to as vertices) $\mathcal{V} = \{v_1, v_2, \dots, v_{|\mathcal{V}|}\}$ and edges $\mathcal{E} = \{(v_i, v_j) \mid i, j \in \{1, 2, \dots, |\mathcal{V}|\}\}$, where each edge (v_i, v_j) represents a connection between nodes v_i and v_j . Both the nodes and edges can be associated with features, denoted by $X = \{x_1, x_2, \dots, x_{|\mathcal{V}|}\}$ and $E = \{e_{ij} \mid (v_i, v_j) \in \mathcal{E}\}$, respectively, which encode information relevant to the entities and structures within the graph. In dynamic graphs, temporal attributes are introduced to capture changes in both node and edge structures over time. These graphs evolve either continuously or discretely as nodes and edges appear, disappear, or update their properties over time. Dynamic graphs are generally categorized into two main types due to their inherent temporal properties. **Discrete-Time Dynamic Graphs** (DTDGs) evolve in distinct time-ordered snapshots $\{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_T\}$, where each graph $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t)$ represents the state of the graph at a specific time-step t . Temporal updates occur at discrete intervals, and each snapshot captures the state of the nodes and edges at that particular time. This formulation is particularly well-suited for applications where data are collected or processed at regular intervals. In contrast, **Continuous-Time Dynamic Graphs** (CTDGs) allow changes to occur continuously over time, with nodes and edges evolving irregularly, and events being associated with specific timestamps. The graph is represented as a function of continuous time $\mathcal{G}(t) = (\mathcal{V}(t), \mathcal{E}(t))$, where t is the timestamp. In this framework, nodes and edges can be updated or interact at any point in time, making CTDGs particularly suitable for modeling asynchronous events.

3.2 Temporal Graph Neural Networks

We follow the standard Temporal Graph Neural Networks (TGNN) paradigm to describe how the GRN works. A typical TGNN learns representations using a dynamic message-passing paradigm, which aggregates information from the neighborhood of each node over

time:

$$\underbrace{v_i^{(k)}}_{\text{update}} \leftarrow \gamma \left(\underbrace{v_i^{(k-1)}}_{\text{update}} \oplus \underbrace{\bigoplus_{j \in \mathcal{N}(i)} \phi \left(\underbrace{v_i^{(k-1)}, v_j^{(k-1)}}_{\text{message}}, e_{ij}, t \right)}_{\text{aggregate}} \right) \quad (1)$$

where $\phi(\cdot)$ is the **message function** that generates messages between nodes v_i and v_j using their features and edge features e_{ij} at time t , $\oplus$ denotes the **aggregation function** that combines messages from all neighboring nodes $j \in \mathcal{N}(i)$, and $\gamma(\cdot)$ is the **update function** that integrates the aggregated messages and the previous state of the node to update its representation over time. This process produces node representations that are subsequently utilized for various downstream tasks.

In this study, we evaluate the efficiency, effectiveness, and scalability of representation learning through two typical tasks in dynamic graph learning: (i) **edge-level prediction**, which predicts whether v_i and v_j are connected at time t , and (ii) **node-level classification**, which infers the state label of v_i at time t .

4 Graph Retention Networks

Graph Retention Networks (GRNs) consist of a stack of multiple GRN blocks, each integrating a multi-scale graph retention module. Within each block, the core computational operator is the graph retention mechanism, which selectively retains and propagates crucial graph information across temporal updates. This retention operator is paired with a feedforward network that refines and transforms the retained graph representations, enabling effective performance on downstream tasks.

4.1 Graph Retention

Graph retention operates as a dual-form graph operator that aggregates information from source and destination nodes in either a parallel or recurrent manner. As described in Section 3.1, consider a destination node v_t^i at time step t , represented by the feature vector $\mathbf{x}_t^i \in \mathbb{R}^d$, and its corresponding set of source nodes (i.e., historical 1-hop neighbors) $\{v_j^j \mid j \in \mathcal{N}(i)\}$. The embeddings of the source nodes $\{\mathbf{x}_t^j \in \mathbb{R}^d \mid j \in \mathcal{N}(i)\}$ and their associated edge attributes $\{e_t^j \in \mathbb{R}^d \mid j \in \mathcal{N}(i)\}$, ordered by interaction time, are organized into matrices $\mathbf{X}^j \in \mathbb{R}^{|\mathcal{N}(i)| \times d}$ and $\mathbf{E}^j \in \mathbb{R}^{|\mathcal{N}(i)| \times d}$, respectively. The time intervals $\{\Delta t_j \mid j \in \mathcal{N}(i)\}$, representing the elapsed time between each interaction and the current time, are collected into a temporal offset vector denoted as $\Delta \mathbf{T}$.

The **message function** is then applied to map the destination node features, edge attributes, and time intervals into a new feature space through a simple linear transformation, denoted as $\phi(\cdot)$. We compute the transformed representation via $\mathbf{X}^j = \phi(\mathbf{X}^j, \mathbf{E}^j, \Delta \mathbf{T}) = \mathbf{X}^j + \mathbf{E}^j \mathbf{W}_e + \text{TE}(\Delta \mathbf{T})$ where $\mathbf{W}_e \in \mathbb{R}^{d \times d}$ is a learnable weight matrix, and $\text{TE}(\cdot)$ represents the temporal encoding function as described in Section 4.2.

Next, we denote graph retention as an **aggregation function** $\oplus(\mathbf{x}_t^i, \mathbf{X}^j)$ that maps $\mathbf{x}_t^i \xrightarrow{\text{agg}} \mathbf{o}_t^i$. We begin by projecting $\mathbf{x}_t^i$ onto a query vector and $\mathbf{X}^j$ onto the key-value pairs as follows:

$$\mathbf{q} = \mathbf{x}_t^i \mathbf{W}_q + \mathbf{b}_q, \quad \mathbf{K} = \mathbf{X}^j \mathbf{W}_k + \mathbf{1}^\top \mathbf{b}_k, \quad \mathbf{V} = \mathbf{X}^j \mathbf{W}_v + \mathbf{1}^\top \mathbf{b}_v, \quad (2)$$

where $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v \in \mathbb{R}^{d \times d}$ are learnable weight matrices, $\mathbf{b}_q, \mathbf{b}_k, \mathbf{b}_v \in \mathbb{R}^d$ are bias vectors, and $\mathbf{1}^\top$ denotes an all-ones column vector of $\mathbb{R}^{|\mathcal{N}(i)| \times 1}$.

Following the idea of retention mechanism computation [31], we maintain a recurrent state $\mathbf{S}^i$ for the source node v_t^i . The output of the graph retention mechanism is then computed in a recurrent manner:

$$\begin{aligned} \mathbf{S}_t^i &= \lambda \mathbf{S}_{t-1}^i + \mathbf{K}_t^\top \mathbf{V}_t, & \mathbf{K}_t &\in \mathbb{R}^{1 \times d}, \mathbf{V}_t \in \mathbb{R}^{1 \times d} \\ \mathbf{o}_t^i &= \mathbf{w}_t \mathbf{q}_t \mathbf{S}_t^i = \mathbf{w}_t \mathbf{q}_t \sum_{k=1}^t \lambda^{t-k} \mathbf{K}_k^\top \mathbf{V}_k, & \mathbf{q}_t &\in \mathbb{R}^{1 \times d} \end{aligned} \quad (3)$$

where the source node is aggregated with the destination node using the state tensor $\mathbf{S}_t^i$, $\mathbf{w}_t$ denotes the edge weight applied at time step t , and $\lambda \in (0, 1)$ is the temporal decay factor to reduce the interference of remote information to current information over time steps. The graph retention formulation is inherently parallelizable within training instances. To accommodate different training and inference scenarios, we reformulate Equations (2) and (3) as:

Parallel Paradigm of Graph Retention. Unlike sequence modeling, node-wise modeling in dynamic graphs excludes self-interactions within the neighboring node set, but rather cross-attention between the source node and its set of neighboring nodes. The parallel graph retention paradigm is formulated as follows:

$$\begin{aligned} \mathbf{Q} &= \mathbf{X}^i \mathbf{W}_q + \mathbf{1}^\top \mathbf{b}_q, \quad \mathbf{K} = \mathbf{X}^j \mathbf{W}_k + \mathbf{1}^\top \mathbf{b}_k, \quad \mathbf{V} = \mathbf{X}^j \mathbf{W}_v + \mathbf{1}^\top \mathbf{b}_v \\ \mathbf{D}_{tk} &= \begin{cases} \lambda^{t-k} \mathbf{w}_t, & t \geq k \\ 0, & t < k \end{cases} \\ \text{GraphRetention}(\mathbf{x}^i, \mathbf{X}^j) &= (\mathbf{Q} \mathbf{K}^\top \odot \mathbf{D}) \mathbf{V} \end{aligned} \quad (4)$$

where $\mathbf{D}$ is a weighted causal mask, with indices $t, k \in \{1, \dots, |\mathcal{N}(i)|\}$ corresponding to the temporal order of source nodes in the neighborhood of v^i . This mask ensures that future source nodes remain invisible to the current target node, thereby preserving the causal structure during computation. The symbol $\odot$ denotes the Hadamard product.

Recurrent Paradigm of Graph Retention. The recurrent graph retention paradigm enables the model to iteratively aggregate information across sequential time steps, making it well-suited for long-term inference in dynamic graphs. At each time-step t , the model updates the recurrent state $\mathbf{S}$ based on the current key-value interaction:

$$\begin{aligned} \mathbf{S}_t &= \lambda \mathbf{S}_{t-1} + \mathbf{K}_t^\top \mathbf{V}_t \\ \text{GraphRetention}(\mathbf{x}_t^i, \mathbf{X}^j) &= \mathbf{w}_t \mathbf{q}_t \mathbf{S}_t, \quad t = 1, \dots, |\mathcal{N}(i)| \end{aligned} \quad (5)$$

This recurrent mechanism enables GRNs to retain and update node embeddings over time at a low computational cost, thereby facilitating the effective learning of complex temporal dependencies.

Chunkwise Paradigm of Graph Retention. To enable efficient training on long-term and dense CTDGs, we utilized a hybrid computational paradigm combining parallel and recurrent approaches. This paradigm splits CTDGs into multiple stages, treating each stage as a distinct training batch, thereby conserving memory by training the model in manageable chunks. The chunkwise graph

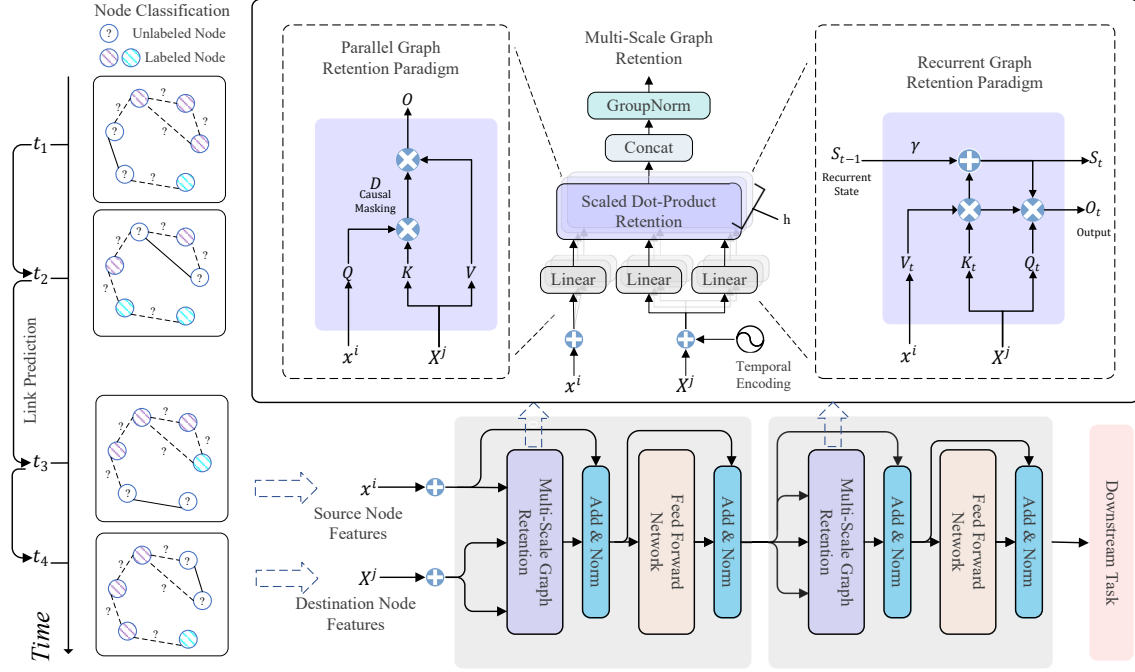


Figure 1: Architecture of GRNs.

retention paradigm for the m -th stage is formulated as follows:

$$\begin{aligned}
 Q_{[m]} &= X_{Bm:B(m+1)}^i W_q + 1^\top b_q, \\
 K_{[m]} &= X_{Bm:B(m+1)}^j W_k + 1^\top b_k, \\
 V_{[m]} &= X_{Bm:B(m+1)}^j W_v + 1^\top b_v, \\
 R_{[m]} &= \lambda^B R_{[m-1]} + K_{[m]}^\top V_{[m]} \\
 \text{GraphRetention}(x^i, X^j) &= \underbrace{(Q_{[m]} K_{[m]}^\top \odot D) V_{[m]}}_{\text{current chunk}} + \underbrace{\lambda^B Q_{[m]} R_{[m-1]}}_{\text{past chunk}}
 \end{aligned} \quad (6)$$

where B is the chunk size and W_q, W_k, W_v are identical to those defined in Equation (2). The chunkwise paradigm without causal masking can also be applied to DTDGs.

Finally, the output of the graph retention mechanism serves as the representation for downstream tasks. The destination node embedding is assigned the output representation via **update function** for the subsequent iteration as $x_{t+1}^j = o_t^j$.

4.2 Temporal Encoding

In this section, we introduce a temporal encoding mechanism from GraphMixer [8] into our architecture. This mechanism enables destination nodes to become time-aware of their neighboring nodes. Considering a specific time interval Δt , we define the time-encoding function as follows:

$$\text{TE}(\Delta t, i) = \cos\left(\Delta t \cdot \sqrt{d}^{-(i-1)/\sqrt{d}}\right) \quad (7)$$

where i is the dimension of the encoding space, and d denotes the overall dimension of the encoding space. Temporal encoding not

only enhances the richness of the feature space but also aligns the model's understanding with the intrinsic temporal characteristics of the underlying graph data.

4.3 Multi-Scale Graph Retention

Multi-Scale Graph Retention (MGR) extends the graph retention mechanism by enabling the model to focus simultaneously on information from multiple representation subspaces across different time intervals. This multi-head approach, inspired by [33], captures complex temporal and relational dynamics, enabling the learning of robust representations in dynamic graphs. In the MGR layer, we use h heads, with each head independently performing graph retention.

$$\begin{aligned}
 \text{head}_i &= \text{GraphRetention}(X^i, X^j) \\
 \text{MGR}(X^i, X^j) &= \text{GN}(\text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h))
 \end{aligned} \quad (8)$$

where each head is associated with distinct decay λ scales and projection parameters $W_q, W_k, W_v \in \mathbb{R}^{d \times d}$. Group normalization (GN) [40] is applied to the concatenated outputs of the heads to stabilize training and maintain numerical precision. This ensures consistency across heads, even when the scales and distributions of the information vary.

Graph Retention Score Normalization. The GRN requires normalization of the graph retention scores to maintain numerical stability within the computational process, i.e., (i) We normalize $QK^\top$ as $QK^\top / \sqrt{d}$; (ii) We replace D with a scaled version $\tilde{D}_{nm}$, where $\tilde{D}_{nm} = D_{nm} / \sum_{i=1}^n D_{ni}$; (iii) Let R denote the retention scores, where $R = QK^\top \odot D$, we further normalize R as

$\tilde{\mathbf{R}}_{nm} = \mathbf{R}_{nm} / \max(|\sum_{i=1}^n \mathbf{R}_{ni}|, 1)$. These operations do not influence the output of graph retention, owing to the GroupNorm.

4.4 Overall Architecture of Graph Retention Networks

Each GRN block contains an MGR layer and a nodewise feedforward network (FFN) module, which together form the core components of the architecture. For an N -layer GRN, N GRN blocks are stacked to construct a deep-learning network capable of capturing complex dependencies and structural nuances in dynamic graphs. The FFN module consists of two linear transformations, with an h-Swish [14] activation gate applied between them to introduce nonlinearity. We provide the overall process of GRNs in Algorithm 1.

For simplicity, we denote the inputs $\{\mathbf{X}^i, \mathbf{X}^j\}$ as $\mathbf{X}$, and the embedding dimension of the FFN is defined as $d_{model} = h \times d$, where h is the number of heads and d is the dimension of each head. The general framework of GRNs is defined as follows:

$$\begin{aligned} H &= \text{MGR}(\text{LN}(\mathbf{X})) + \mathbf{X} \\ O &= \text{FFN}(\text{LN}(H)) + H \end{aligned} \quad (9)$$

where $\text{FFN}(x) = (\text{hSwish}(xW_1))W_2$

where $\text{hSwish}(x) = x \cdot \text{ReLU6}(x+3)/6 = x \cdot \min(\max(0, x+3), 6)/6$, $\text{LN}(\cdot)$ represents layer normalization [2], and W_1, W_2 are learnable weight matrices in the FFN used for integrating the multiple retention outputs generated by the MGR.

5 Experiments

5.1 Setup

Datasets. We conduct experiments on thirteen benchmark datasets (i.e., Wikipedia, Reddit, MOOC, LastFM, Myket, Enron, UCI, Flights, Can. Parl., US Legis., UN Trade, UN Vote, and Contact) from various domains, as collected by [27]. The statistics of the datasets are summarized in Table 3, with more details provided in Appendix A.2.

Baselines. We compare the GRN with nine strong baselines: JODIE [16], DyRep [32], TGAT [42], TGN [29], CAWN [37], EdgeBank [27], TCL [36], GraphMixer [8], and DyGFormer [45]. These baselines encompass a variety of architectures, including GNNs, memory-based methods, transformers, and random walk approaches. While these models utilize truncation and sampling strategies to aggregate past information, our model retains a recurrent state for graph retention. The sophisticated designs of these baselines reflect current advancements in improving representation learning for dynamic graphs. Detailed descriptions of the baselines are provided in Appendix A.3.

Downstream Tasks and Evaluation Metrics. We evaluate our model on edge-level prediction and node-level classification tasks. Specifically, we integrate our model with a link predictor and node classifier as downstream components. These components consist of multiple fully connected layers, with the link predictor estimating the probability of a connection between two nodes and the node classifier predicting the probability of a node belonging to a specific state label. In addition, we evaluate the edge-level prediction task under both transductive and inductive settings. In dynamic graphs, transductive learning involves observing node and edge information during training, allowing the model to make

predictions based on a known structure. In contrast, inductive learning requires the model to predict links between nodes that remain unobserved during training. For performance evaluation, we use Average Precision (AP) to assess model precision and the Area Under the Receiver Operating Characteristic Curve (AUC-ROC) to evaluate model sensitivity and robustness for both prediction and classification tasks. To measure training and inference efficiency, we compared models based on training and inference latency, GPU memory consumption, and throughput.

Implementation and Configurations. We follow the standard training pipeline and evaluation protocol proposed by the Dynamic Graph Library (DyGLib)¹ and the Temporal Graph Benchmark (TGB)² to implement our model to ensure consistency of comparisons. Nevertheless, we clarify in Appendix A.5 that our implementation has some inconsistencies with it. The chunk-wise paradigm is used for training, while the recurrent computation paradigm is adopted for fast inference to ensure the retention of identical long-term dependencies. For baseline models, we use the optimal configurations reported by DyGLib after validation, while the optimal configurations for our model and computer resources for implementation are provided in Appendix A.4. The implementation code is available at [\[Code\]](#).

5.2 Performance Comparison with Baselines

This section gives the results of comparative experiments evaluating the *effectiveness*, *efficiency*, and *scalability* of GRN, with further details provided in Appendix B. We report results (i) on both link prediction and node classification tasks and (ii) under transductive and inductive settings. Additionally, we assess training and inference efficiency (iii) by measuring latency, GPU memory consumption, and throughput.

Efficiency. *The GRN enjoys more efficient training and inference in dynamic graph representation learning.* We evaluate the efficiency of training and inference in Figure 2. During training, the GRN exhibits latency reductions ranging from 1.13x to 15.99x, GPU memory consumption reductions of 1.53x to 3.55x, and throughput improvements of 1.17x to 16.61x compared to baseline models. The differences in inference efficiency are even more pronounced, with the GRN achieving at least a 6.1x reduction in latency, 1.66x lower GPU usage, and up to an 86.7x increase in throughput. These improvements are grounded in two architectural properties: (i) The GRN adopts a unified state-based design that obviates the need for explicit memory modules and a single recurrent state tensor suffices to preserve long-term temporal dependencies. (ii) The GRN leverages a dual-paradigm execution strategy: the parallelism-enabled chunkwise mode significantly reduces training memory overhead, while the recurrent mode allows rapid inference with minimal synchronization. A formal analysis of time and space complexity of the GRNs is provided in Appendix A.1.

Effectiveness. *The GRN achieves strong and comprehensive performance in both transductive and inductive settings* (see Tables 1, 4, 5, and 6), as well as on link prediction and node classification tasks (see Table 7). The results highlight the GRN’s robustness to a wide range of temporal graph structures. On CTDGs such as

¹<https://github.com/yule-BUAA/DyGLib>

²<https://github.com/shenyangHuang/TGB>

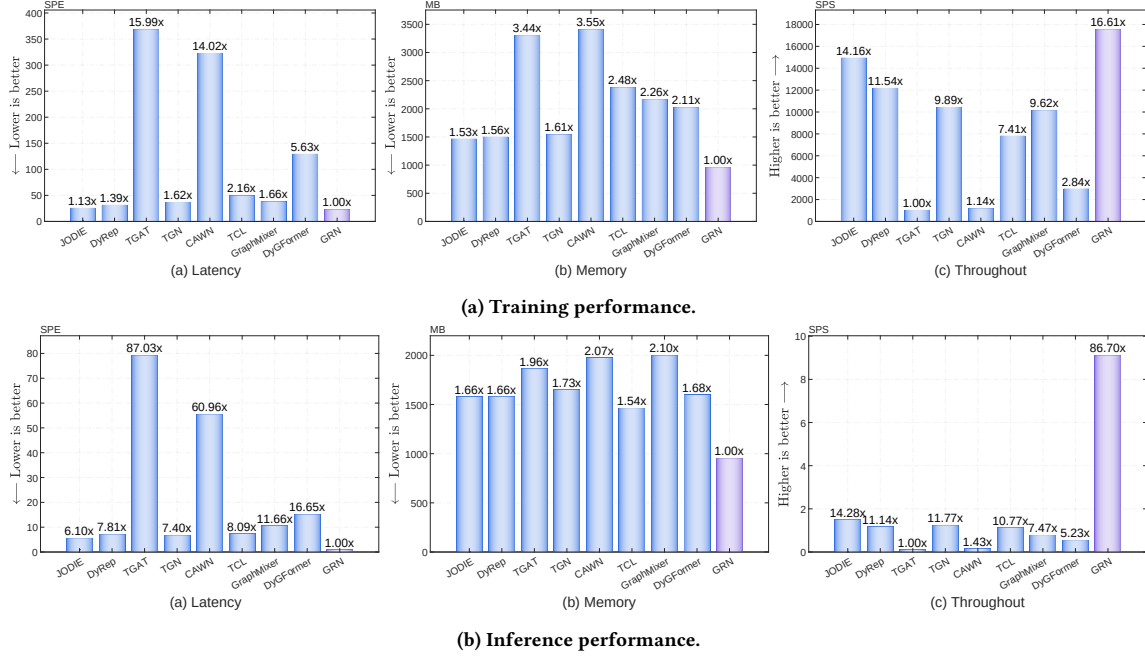


Figure 2: Training and inference efficiency comparison. SPE = Seconds Per Epoch; MB = MegaByte; SPS = Samples Per Second. Subfigure (a) compares performance of baseline with that of GRNs when trained in chunkwise paradigm, and subfigure (b) compares performance of GRNs inference in recurrent paradigm with that of baselines. All multipliers are based on model with the lowest value.

Table 1: AP for transductive link prediction on dynamic graphs with random negative sampling strategy. The best performances are highlighted in bold; second-best results are underlined.

Datasets	JODIE	DyRep	TGAT	TGN	CAWN	EdgeBank	TCL	GraphMixer	DyGFormer	GRN
Wikipedia	91.22±3.73	90.94±0.48	94.00±0.31	97.50±0.06	<u>98.41±0.02</u>	90.37±0.00	95.18±0.14	95.91±0.05	98.86±0.04	97.49±0.35
Reddit	97.61±0.17	97.44±0.12	96.64±0.04	96.96±0.16	98.79±0.02	94.71±0.00	93.25±0.18	95.41±0.04	98.98±0.06	99.50±0.03
MOOC	75.66±5.05	68.74±1.33	<u>77.73±1.23</u>	74.73±1.65	68.42±1.25	57.97±0.00	75.19±1.34	74.23±0.40	72.12±1.16	91.63±1.18
LastFM	69.05±0.74	69.23±1.69	64.05±0.40	56.13±2.82	<u>83.09±0.05</u>	79.29±0.00	59.33±0.22	69.64±0.03	90.05±0.03	72.93±0.62
Myket	87.67±0.08	<u>87.49±0.09</u>	65.54±5.14	83.04±1.81	85.40±0.19	57.31±0.00	68.91±2.66	86.55±0.01	79.91±0.46	82.94±2.51
Enron	78.38±1.24	71.71±3.02	67.09±0.53	74.41±1.56	<u>86.01±0.66</u>	83.53±0.00	69.37±1.55	80.99±0.11	87.42±2.09	83.15±2.49
UCI	80.38±2.39	52.14±2.64	78.88±0.54	87.15±0.93	<u>92.88±0.21</u>	76.20±0.00	85.13±1.02	91.91±0.70	95.29±0.05	80.55±1.38
Flights	92.38±0.64	90.22±0.78	90.96±0.03	92.67±1.60	<u>96.88±0.23</u>	89.35±0.00	89.59±0.17	89.69±0.01	98.72±0.03	96.63±1.46
Can. Parl.	73.18±0.77	67.08±0.93	67.51±1.09	68.84±1.03	67.67±2.70	64.55±0.00	66.04±2.46	75.37±0.71	94.49±0.92	91.91±0.23
US Legis.	<u>78.45±0.35</u>	68.72±3.28	61.36±3.91	70.91±0.23	66.16±1.42	58.39±0.00	60.38±0.56	70.16±0.56	72.42±0.40	79.21±3.82
UN Trade	<u>65.26±0.67</u>	63.67±1.13	59.97±0.78	60.90±0.66	61.76±0.15	60.41±0.00	60.20±0.33	54.19±4.33	57.04±1.30	71.54±0.68
UN Vote	<u>63.12±1.41</u>	60.17±0.71	52.13±0.42	57.15±1.09	51.82±0.15	58.49±0.00	51.35±0.13	51.45±0.23	53.13±0.11	83.09±1.33
Contact	93.04±0.23	84.66±2.48	<u>93.42±0.17</u>	91.92±2.05	87.26±0.20	92.58±0.00	89.01±0.50	89.28±0.31	97.89±0.05	89.34±0.75
Avg. Rank	4.15	6.54	6.85	5.23	4.62	7.54	7.92	5.92	<u>3.15</u>	3.08

Wikipedia, Reddit, and MOOC, the GRN consistently ranks among the top performers, demonstrating its ability to model irregular and fine-grained temporal patterns. It also excels on long-term and sparse graphs like UN Vote and US Legis., where balancing long-term and short-term dependencies is critical. In contrast, for dense graphs with frequent short-interval interactions such as Contact and Flights, the GRN maintains competitive performance, showcasing its scalability and efficiency. Moreover, its strong results on imbalanced node classification tasks with limited features (see Appendix B.2) further attest to its architectural generality. These

empirical gains stem from three key factors: (i) the recurrent state S captures long-term temporal dependencies, while the decay factor λ conjugates with it to allow information decay, enhancing temporal sensitivity (see Section 5.3); (ii) the graph retention mechanism yields a smooth optimization landscape conducive to generalization (see Figure 3); and (iii) the unified design supports versatile learning across both DTDGs and CTDGs without requiring model reconfiguration (see Section 5.4). However, we find some performance differences between the GRN and feature-enhanced baselines on relatively dense graphs (e.g. LastFM and Contact). Unlike methods

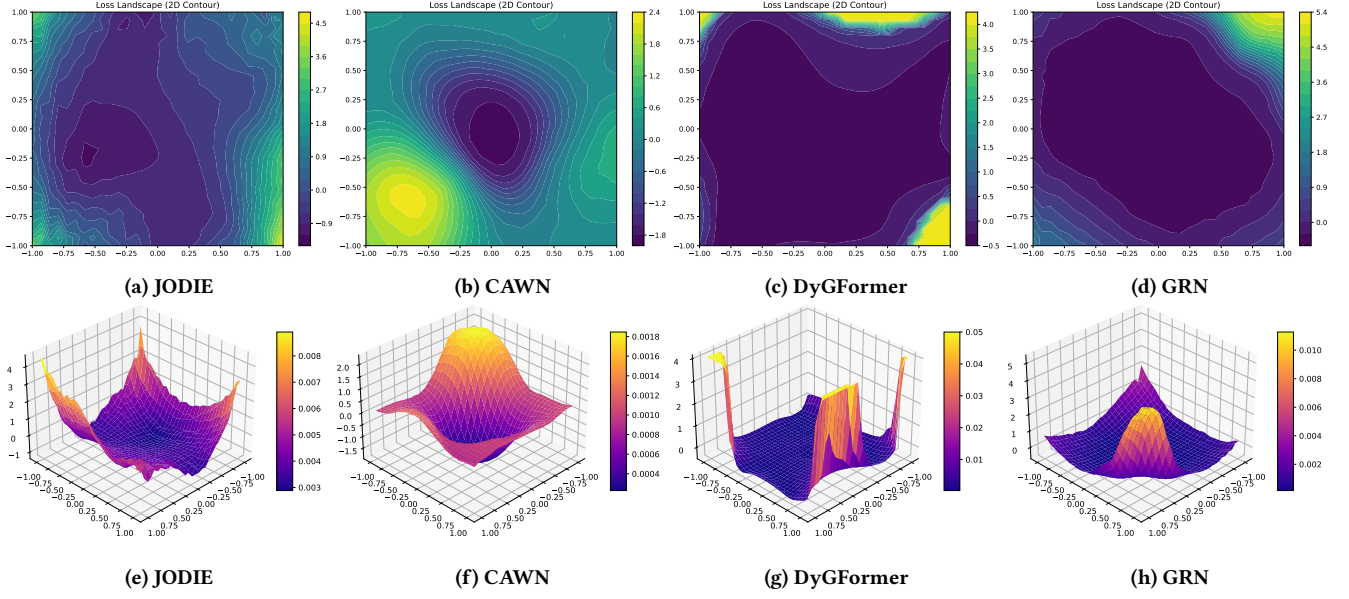


Figure 3: 2D (Filled Contour) and 3D (Surface) loss landscape on Wikipedia dataset. We show the top-ranked performers here, and additional loss landscapes for all baselines are in Appendix B.3.

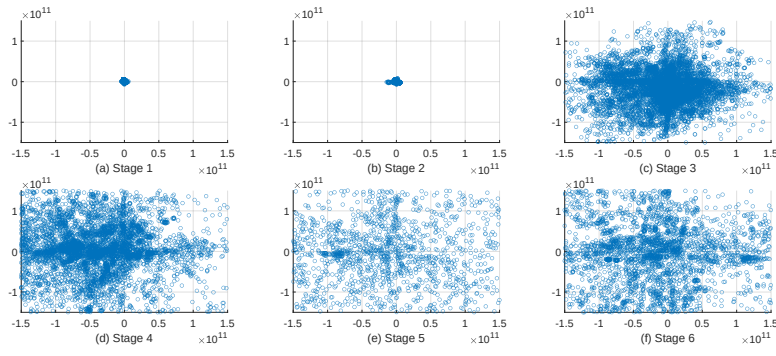


Figure 4: T-SNE visualization of recurrent states with $\lambda = 1 - 2^{-6}$. Each stage represents the same 5% interval of the input data within a training epoch.

that improve performance through input-level augmentation (e.g., neighbor co-occurrence encoding and patching in DyGFormer), GRN is designed as a unified, end-to-end architecture that learns directly from raw input features. To better understand the performance gap, we conduct an exploratory comparison by applying the identical feature enhancement strategies to GRN. As shown in Table 2, this variant (denoted as GRN+) narrows the gap, achieving up to +23.31% improvement on transductive LastFM and consistent gains across both evaluation settings. **Scalability.** The proposed GRN architecture demonstrates strong scalability, as evidenced by its consistent superiority across all experimental settings in both *efficiency* and *effectiveness*. To better understand its generalizability, we visualize the 2D (filled contour) and 3D (surface) loss landscapes of the top-performing models on the Wikipedia dataset in Figure 3.

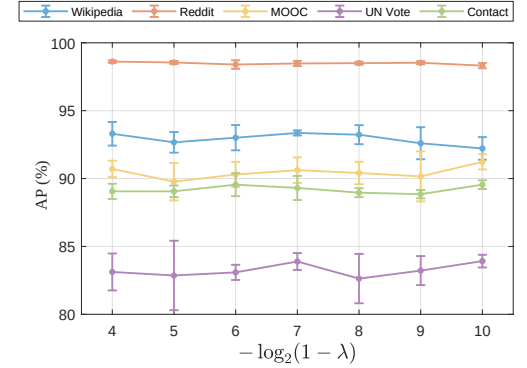


Figure 5: Impact of decay strength with 95% CI on AP across five datasets under transductive setting.

Table 2: Comparison of predictive performance with feature enhancement (GRN+).

Setting	Dataset	CAWN	GraphMixer	DyGFormer	GRN	GRN+	Improvement
Transductive	LastFM	83.09±0.05	69.64±0.03	90.05±0.03	72.93±0.62	89.93±0.05	+23.31%
	Contact	87.26±0.20	89.28±0.31	97.89±0.05	89.34±0.75	97.93±0.08	+09.61%
Inductive	LastFM	87.55±0.09	81.63±0.20	93.11±0.03	83.44±0.95	91.04±0.15	+09.11%
	Contact	86.36±0.31	87.46±0.31	97.71±0.04	91.18±1.12	98.09±0.03	+07.58%

Compared to competitive baselines, the GRN displays a distinctly flatter and broader minimum in both 2D and 3D spaces. Additional results on Reddit and MOOC datasets are provided in Appendix B.3, which further substantiate the generalization advantage of GRNs under varied graph densities and event granularities.

5.3 Why is Graph Retention Well-Suited for Dynamic Graph Modeling?

While comparative experiments on the efficiency, effectiveness, and scalability of GRNs demonstrate its excellent ability to model dynamic graphs, we further discuss *why it is well-suited for dynamic graph modeling* in terms of its core graph retention operator.

Recurrent states contribute to dynamic graph representation learning. To better understand how recurrent states contribute to dynamic graph representation learning, we perform detailed t-SNE visualizations of recurrent states over six chronological stages under different scale decay factors in Figures 4 and 7. Across all settings, we observe a consistent dynamic unfolding of the latent states. At early stages (Stage 1–2), recurrent states are tightly clustered at the presented order of magnitude, indicating relatively minimal temporal differentiation. As time progresses (Stage 3–6), the state representations gradually disperse and form more structured and heterogeneous clusters. This expansion clearly reflects the accumulation of temporal signals, highlighting the ability of the recurrent state to encode long-term interactions. The visualizations provide strong empirical evidence that the recurrent state encourages temporally smooth yet discriminative latent dynamics, supporting the GRN’s ability to generalize.

Multi-scale decay balances long-term and short-term dependencies. The multi-scale decay mechanism in GRNs enables the model to flexibly adapt to datasets with highly diverse temporal dynamics. As shown in Figures 5 and 8, AP scores vary with different decay levels across both transductive and inductive link prediction tasks on most datasets. For datasets characterized by dense and rapidly evolving interactions (Contact and MOOC), stronger decay generally leads to better performance by emphasizing recent events and reducing the influence of outdated information. In contrast, for datasets with sparse or long-term temporal dependencies (UN Vote), weaker decay is often more effective, allowing the model to retain and leverage long-term temporal signals. These results further underscore the benefit of multi-head temporal modeling, where assigning distinct decay factors to each head allows GRNs to concurrently capture both short-term and long-term patterns.

5.4 Ablation Study

To evaluate the contribution of individual components within the architecture, we ablate various design choices of GRN. To isolate the effect of each design component, we construct the following GRN variants:

- w/o S recurrent state: removes the recurrent temporal memory module, forcing the model to rely solely on instantaneous observations.
- w/o λ decay: disables the temporal decay operator, treating all past interactions equally regardless of recency (i.e. $\lambda = 1$).
- w/o multi-scale decay: replaces adaptive decay with a fixed, single-scale decay, reducing flexibility across datasets.
- w/o temporal encoding: removes time encoding, making the model invariant to the order and spacing of temporal events.

The results in Figure 6 offer a systematic breakdown of each architectural component’s contribution to the GRN under both transductive and inductive link prediction settings on the Wikipedia

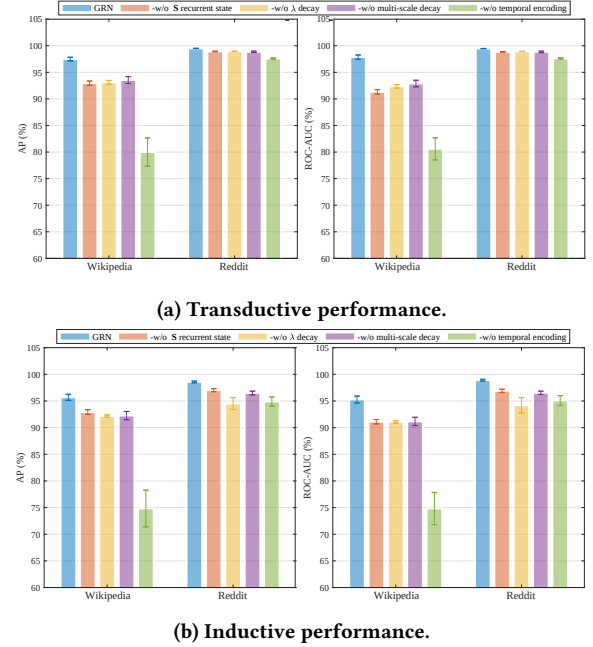


Figure 6: Results of ablation experiments on GRN components.

and Reddit datasets. Overall, the ablation study confirms the complementarity and necessity of all architectural components. The recurrent state S enables long-term memory, the decay module provides time-sensitive signal weighting, and temporal encoding allows the model to resolve precise temporal order. Their synergy is essential for achieving strong generalization in both seen (transductive) and unseen (inductive) dynamic graph settings. Detailed analysis is provided in Appendix B.5

6 Conclusion

In this work, we propose the GRN as a unified and scalable architecture that incorporates a novel graph retention mechanism to effectively address key challenges in dynamic graph learning. Extensive experiments across a wide range of dynamic graph tasks and settings confirm its high efficiency, comprehensive effectiveness, and strong scalability, showcasing its robustness and versatility across various dynamic graph tasks. We believe this work opens up promising directions for further exploration in dynamic graph representation learning. Concerns and limitations are discussed in Appendix C, which we aim to address in future work.

ACKNOWLEDGMENTS

This project was supported by the NZ EQUALS program (4260/76892); the Post-funded Key Project of National Social Science Fund of China (25FGLA001); the Ministry of Education of Humanities and Social Science Project of China (25YJA630043); the Natural Science Foundation of Hubei Province of China (JCZRYB202401041); the National Natural Science Foundation of China (71974069); the Program of China Scholarship Council (202408250021 and 202506770023).

References

- [1] Unai Alvarez-Rodriguez, Federico Battiston, Guilherme Ferraz de Arruda, Yamil Moreno, Matjaž Perc, and Vito Latora. 2021. Evolutionary dynamics of higher-order interactions in social networks. *Nature Human Behaviour* 5, 5 (2021), 586–595.
- [2] Jimmy Lei Ba. 2016. Layer normalization. *arXiv preprint arXiv:1607.06450* (2016).
- [3] Guangji Bai, Chen Ling, and Liang Zhao. 2022. Temporal domain generalization with drift-aware dynamic neural networks. *arXiv preprint arXiv:2205.10664* (2022).
- [4] Claudio DT Barros, Matheus RF Mendonça, Alex B Vieira, and Artur Ziviani. 2021. A survey on embedding dynamic graphs. *ACM Computing Surveys (CSUR)* 55, 1 (2021), 1–37.
- [5] Peter Battaglia, Razvan Pascanu, Matthew Lai, Danilo Jimenez Rezende, et al. 2016. Interaction networks for learning about objects, relations and physics. *Advances in neural information processing systems* 29 (2016).
- [6] Maciej Besta, Marc Fischer, Vasiliki Kalavri, Michael Kapralov, and Torsten Hoefler. 2019. Practice of streaming processing of dynamic graphs: Concepts, models, and systems. *arXiv preprint arXiv:1912.12740* (2019).
- [7] Stephen Bonner, Amir Atapour-Abarghouei, Philip T Jackson, John Brennan, Ibad Kureshi, Georgios Theodoropoulos, Andrew Stephen McGough, and Boguslaw Obara. 2019. Temporal neighbourhood aggregation: Predicting future links in temporal graphs via recurrent variational graph convolutions. In *2019 IEEE international conference on big data (Big Data)*. IEEE, 5336–5345.
- [8] Weilin Cong, Si Zhang, Jian Kang, Baichuan Yuan, Hao Wu, Xin Zhou, Hanghang Tong, and Mehrdad Mahdavi. 2023. Do we really need complicated model architectures for temporal networks? *arXiv preprint arXiv:2302.11636* (2023).
- [9] Peng Cui, Xiao Wang, Jian Pei, and Wenwu Zhu. 2018. A survey on network embedding. *IEEE transactions on knowledge and data engineering* 31 (2018), 833–852.
- [10] ZhengZhao Feng, Rui Wang, TianXing Wang, Mingli Song, Sai Wu, and Shuibing He. 2024. A Comprehensive Survey of Dynamic Graph Neural Networks: Models, Frameworks, Benchmarks, Experiments and Challenges. *arXiv preprint arXiv:2405.00476* (2024).
- [11] Shihong Gao, Yiming Li, Yanyan Shen, Yingxia Shao, and Lei Chen. 2024. ETC: Efficient Training of Temporal Graph Neural Networks over Large-scale Dynamic Graphs. *Proceedings of the VLDB Endowment* 17 (2024), 1060–1072.
- [12] Mingyu Guan, Anand Padmanabha Iyer, and Taesoo Kim. 2022. DynaGraph: dynamic graph neural networks at scale. In *Proceedings of the 5th ACM SIGMOD joint International Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*. 1–10.
- [13] Shengnan Guo, Youfang Lin, Huaiyu Wan, Xiucheng Li, and Gao Cong. 2021. Learning dynamics and heterogeneity of spatial-temporal graph data for traffic forecasting. *IEEE Transactions on Knowledge and Data Engineering* 34, 11 (2021), 5415–5428.
- [14] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, et al. 2019. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF international conference on computer vision*. 1314–1324.
- [15] Shenyang Huang, Farimah Poursafaei, Jacob Danovitch, Matthias Fey, Weihua Hu, Emanuele Rossi, Jure Leskovec, Michael Bronstein, Guillaume Rabusseau, and Reihaneh Rabbany. 2024. Temporal graph benchmark for machine learning on temporal graphs. *Advances in Neural Information Processing Systems* 36 (2024).
- [16] Srikanth Kumar, Xikun Zhang, and Jure Leskovec. 2019. Predicting dynamic embedding trajectory in temporal interaction networks. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 1269–1278.
- [17] Hanjie Li, Changsheng Li, Kaituo Feng, Ye Yuan, Guoren Wang, and Hongyuan Zha. 2024. Robust knowledge adaptation for dynamic graph neural networks. *IEEE Transactions on Knowledge and Data Engineering* (2024).
- [18] Jie Li and Matteo Convertino. 2021. Inferring ecosystem networks as information flows. *Scientific reports* 11, 1 (2021), 7094.
- [19] Jintang Li, Zhouxin Yu, Zulun Zhu, Liang Chen, Qi Yu, Zibin Zheng, Sheng Tian, Ruofan Wu, and Changhua Meng. 2023. Scaling up dynamic graph representation learning via spiking neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 8588–8596.
- [20] Yuhong Luo and Pan Li. 2024. No Need to Look Back: An Efficient and Scalable Approach for Temporal Graph Representation Learning. *arXiv preprint arXiv:2402.01964* (2024).
- [21] Franco Manessi, Alessandro Rozza, and Mario Manzo. 2020. Dynamic graph convolutional networks. *Pattern Recognition* 97 (2020), 107000.
- [22] John A Miller, Mohammed Aldosari, Farah Saeed, Nasid Habib Barna, Subas Rana, I Budak Arpinar, and Ninghao Liu. 2024. A survey of deep learning and foundation models for time series forecasting. *arXiv preprint arXiv:2401.13912* (2024).
- [23] Shengjie Min, Zhan Gao, Jing Peng, Liang Wang, Ke Qin, and Bo Fang. 2021. Stgns—a spatial-temporal graph neural network framework for time-evolving social networks. *Knowledge-Based Systems* 214 (2021), 106746.
- [24] Jayanta Mondal and Amol Deshpande. 2012. Managing large dynamic graphs efficiently. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*. 145–156.
- [25] Mandani Ntekeuli, Gerasimos Spanakis, Lourens Waldorp, and Anne Roefs. 2024. Exploiting Individual Graph Structures to Enhance Ecological Momentary Assessment (EMA) Forecasting. In *2024 IEEE 40th International Conference on Data Engineering Workshops (ICDEW)*. IEEE, 158–166.
- [26] Aldo Pareja, Giacomo Domeniconi, Jie Chen, Tengfei Ma, Toyotaro Suzumura, Hiroki Kanezashi, Tim Kaler, Tao Schardl, and Charles Leiserson. 2020. EvolveGCN: Evolving graph convolutional networks for dynamic graphs. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 5363–5370.
- [27] Farimah Poursafaei, Shenyang Huang, Kellin Pelrine, and Reihaneh Rabbany. 2022. Towards better evaluation for dynamic link prediction. *Advances in Neural Information Processing Systems* 35 (2022), 32928–32941.
- [28] Xiafei Qiu, Wubin Cen, Zhengping Qian, You Peng, Ying Zhang, Xuemin Lin, and Jingren Zhou. 2018. Real-time constrained cycle detection in large dynamic graphs. *Proceedings of the VLDB Endowment* 11, 12 (2018), 1876–1888.
- [29] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. 2020. Temporal graph networks for deep learning on dynamic graphs. *arXiv preprint arXiv:2006.10637* (2020).
- [30] Junwei Su, Difan Zou, and Chuan Wu. 2024. PRES: Toward Scalable Memory-Based Dynamic Graph Neural Networks. *arXiv preprint arXiv:2402.04284* (2024).
- [31] Yutao Sun, Li Dong, Shaohan Huang, Shuming Xia, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. 2023. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621* (2023).
- [32] Rakshit Trivedi, Mehrdad Farajtabar, Prasenjeet Biswal, and Hongyuan Zha. 2019. Dyrep: Learning representations over dynamic graphs. In *International conference on learning representations*.
- [33] A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).
- [34] Jana Vatter, Ruben Mayer, and Hans-Arno Jacobsen. 2023. The evolution of distributed systems for graph neural networks and their origin in graph processing and deep learning: A survey. *Comput. Surveys* 56, 1 (2023), 1–37.
- [35] Xinchun Wan, Kaiqiang Xu, Xudong Liao, Yilun Jin, Kai Chen, and Xin Jin. 2023. Scalable and efficient full-graph gnn training for large graphs. *Proceedings of the ACM on Management of Data* 1 (2023), 1–23.
- [36] Lu Wang, Xiaofu Chang, Shuang Li, Yunfei Chu, Hui Li, Wei Zhang, Xiaofeng He, Le Song, Jingren Zhou, and Hongxia Yang. 2021. TcL: Transformer-based dynamic graph modelling via contrastive learning. *arXiv preprint arXiv:2105.07944* (2021).
- [37] Yanbang Wang, Yen-Yu Chang, Yunyu Liu, Jure Leskovec, and Pan Li. 2021. Inductive representation learning in temporal networks via causal anonymous walks. *arXiv preprint arXiv:2101.05974* (2021).
- [38] Jason Weston, Sumit Chopra, and Antoine Bordes. 2014. Memory networks. *arXiv preprint arXiv:1410.3916* (2014).
- [39] Yuxia Wu, Yuan Fang, and Lizi Liao. 2024. On the Feasibility of Simple Transformer for Dynamic Graph Modeling. In *Proceedings of the ACM on Web Conference 2024*. 870–880.
- [40] Yuxin Wu and Kaiming He. 2018. Group normalization. In *Proceedings of the European conference on computer vision (ECCV)*. 3–19.
- [41] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, and Chengqi Zhang. 2019. Graph wavenet for deep spatial-temporal graph modeling. *arXiv preprint arXiv:1906.00121* (2019).
- [42] Da Xu, Chuanwei Ruan, Evren Korpeoglu, Sushant Kumar, and Kannan Achan. 2020. Inductive representation learning on temporal graphs. *arXiv preprint arXiv:2002.07962* (2020).
- [43] Nan Yin, Mengzhu Wang, Zhenghan Chen, Giulia De Masi, Huan Xiong, and Bin Gu. 2024. Dynamic spiking graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 16495–16503.
- [44] Bing Yu, Haoteng Yin, and Zhanxing Zhu. 2017. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. *arXiv preprint arXiv:1709.04875* (2017).
- [45] Le Yu, Leilei Sun, Bowen Du, and Weifeng Lv. 2023. Towards better dynamic graph learning: New architecture and unified library. *Advances in Neural Information Processing Systems* 36 (2023), 67686–67700.
- [46] Siwei Zhang, Xi Chen, Yun Xiong, Xixi Wu, Yao Zhang, Yongrui Fu, Yinglong Zhao, and Jiawei Zhang. 2024. Towards adaptive neighborhood for advancing temporal interaction graph modeling. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4290–4301.
- [47] Zeyang Zhang, Xin Wang, Ziwei Zhang, Haoyang Li, Yijian Qin, Simin Wu, and Wenwu Zhu. 2023. LLM4DyG: Can Large Language Models Solve Problems on Dynamic Graphs? *arXiv preprint arXiv:2310.17110* (2023).
- [48] Hongkuan Zhou, Da Zheng, Israt Nisa, Vasileios Ioannidis, Xiang Song, and George Karypis. 2022. Tgl: A general framework for temporal gnn training on billion-scale graphs. *arXiv preprint arXiv:2203.14883* (2022).
- [49] Hongkuan Zhou, Da Zheng, Xiang Song, George Karypis, and Viktor Prasanna. 2023. Distgl: Distributed memory-based temporal graph neural network training. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–12.

- [50] Yangjie Zhou, Jingwen Leng, Yaoxu Song, Shuwen Lu, Mian Wang, Chao Li, Minyi Guo, Wenting Shen, Yong Li, Wei Lin, et al. 2023. Ugrapher: High-performance graph operator computation via unified abstraction for graph neural networks. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 878–891.
- [51] Dingyuan Zhu, Peng Cui, Ziwei Zhang, Jian Pei, and Wenwu Zhu. 2018. High-order proximity preserved embedding for dynamic networks. *IEEE Transactions on Knowledge and Data Engineering* 30, 11 (2018), 2134–2144.

A Additional Descriptions

A.1 Time and Space Complexity Analysis

Algorithm 1: Overall process of GRNs

Input: Dynamic graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, node features $\mathbf{X}$, edge attributes $\mathbf{E}$, time intervals ΔT

Output: Node representations $\mathbf{O}$

- 1 **Initialize:** All trainable parameters $\mathbf{W}, \mathbf{b}$ for each head h ;
Recurrent state $\mathbf{S}_0$;
- 2 **for** $t = 1$ **to** T **do**
- 3 **for each node** v_t^i (destination node) **do**
- 4 $\mathbf{X}_t^j \leftarrow \text{Message}(\mathbf{X}_t^j, \mathbf{E}_t^j, \Delta T_t)$;
- 5 **for each head** $h = 1, \dots, H$ **do**
- 6 # Compute $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ with head-specific parameters;
- 7 $\mathbf{Q}_{t,h} = \mathbf{x}_t^i \mathbf{W}_{q,h} + \mathbf{1}^\top \mathbf{b}_{q,h}$;
- 8 $\mathbf{K}_{t,h} = \mathbf{x}_t^j \mathbf{W}_{k,h} + \mathbf{1}^\top \mathbf{b}_{k,h}$;
- 9 $\mathbf{V}_{t,h} = \mathbf{x}_t^j \mathbf{W}_{v,h} + \mathbf{1}^\top \mathbf{b}_{v,h}$
- 10 # Compute graph retention output for each head;
- 11 $\mathbf{o}_{t,h}^i = \text{GraphRetention}(\mathbf{Q}_{t,h}, \mathbf{K}_{t,h}, \mathbf{V}_{t,h}, \mathbf{S}_{t-1})$
- 12 **end**
- 13 **end**
- 14 # Concatenate all head outputs;
- 15 $\mathbf{H} = \text{Concatenate}(\mathbf{o}_{t,1}^i, \dots, \mathbf{o}_{t,H}^i)$;
- 16 # Normalize the concatenated output;
- 17 $\mathbf{H} \leftarrow \text{GroupNorm}(\mathbf{H})$;
- 18 # Feed forward;
- 19 $\mathbf{O} = \text{FFN}(\mathbf{H})$
- 20 # Update destination nodes' features;
- 21 $\mathbf{X}_{t+1} \leftarrow \mathbf{O}$
- 22 **end**

The computational efficiency of GRN arises from its flexible retention paradigms and parallelizable operations. Below, we analyze the time and space complexity of each core component.

Message and Temporal Encoding. Each node aggregates temporal messages from its historical neighbors. The message function requires $O(d)$ operations per edge for linear transformations and temporal encoding. Given $|E_t|$ edges at time t , this step incurs a total cost of $O(|E_t| \cdot d)$ per time step.

Graph Retention Operator. For a destination node with $|\mathcal{N}(i)|$ historical neighbors:

- In the parallel paradigm, computing $\mathbf{QK}^\top$ where $\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{|\mathcal{N}(i)| \times d}$ costs $O(|\mathcal{N}(i)|^2 \cdot d)$, followed by multiplication with $\mathbf{V}$. Across all nodes, the total complexity is $O(|V_t| \cdot \bar{N}^2 \cdot d)$.

- In the recurrent paradigm, the state $\mathbf{S}_t \in \mathbb{R}^{d \times d}$ is updated via $\mathbf{S}_t = \lambda \mathbf{S}_{t-1} + \mathbf{K}_t^\top \mathbf{V}_t$. Each update costs $O(d^2)$, yielding $O(|V_t| \cdot \bar{N} \cdot d^2)$ in total.
- The chunkwise paradigm processes neighbors in chunks of size B . Intra-chunk attention costs $O(B^2 \cdot d)$, cross-chunk state costs $O(B \cdot d^2)$. Total: $O(|V_t| \cdot \bar{N} \cdot (B \cdot d + d^2))$, with peak memory reduced to $O(B^2 + d^2)$.

Multi-Scale Graph Retention. Each of H heads executes graph retention independently: $O(H \cdot \bar{N}^2 \cdot d)$ (parallel) or $O(H \cdot \bar{N} \cdot d^2)$ (recurrent). Group normalization adds $O(H \cdot d)$.

Feedforward Network (FFN). Two linear layers with $d_{\text{model}} = H \cdot d$. Complexity: $O(d_{\text{model}}^2)$ per node.

Overall Complexity. With L layers, total time complexity per step: $O(L \cdot |V_t| \cdot (H \cdot \bar{N}^2 \cdot d + d_{\text{model}}^2))$ (parallel); $O(L \cdot |V_t| \cdot (H \cdot \bar{N} \cdot d^2 + d_{\text{model}}^2))$ (recurrent). Space complexity: $O(|V_t| \cdot L \cdot H \cdot d^2 + |V_t| \cdot d_{\text{model}} + |E_t| \cdot d)$. The recurrent paradigm is advantageous when $\bar{N} > d$, avoiding $O(\bar{N}^2)$ attention maps for long-range graphs.

A.2 Additional Descriptions of Datasets

The statistics for the datasets used are summarized in Table 3.

A.3 Descriptions of Baselines

The details of the nine baselines are described as follows:

- **JODIE:** Designed for temporal bipartite networks such as user-item interactions, JODIE utilizes two coupled recurrent neural networks to dynamically update the states of users and items upon interactions. A projection mechanism predicts future user/item embeddings by learning their temporal trajectories.
- **TGAT:** Utilizes graph attention mechanisms combined with temporal encoding to model node dynamics over time. This approach ensures effective representation learning by combining temporal and structural graph information.
- **TGN:** Incorporates a memory module for each node to maintain evolving states, which are updated through a message-passing paradigm consisting of a message function, aggregator, and updater. An embedding module generates temporal node representations.
- **CAWN:** Employs causal Anonymous Walks to capture causal patterns in dynamic graphs. Each walk is encoded using recurrent neural networks, and the results are aggregated to form node representations. This method effectively leverages temporal and causal graph structures.
- **EdgeBank:** A memory-based model for dynamic link prediction, it retains observed edges in memory and predicts interactions based on past edge information. Variants such as EdgeBank_∞ and $\text{EdgeBank}_{\text{tw}}$ adapt memory retention to different temporal contexts, such as fixed-size windows or edge appearance thresholds.
- **TCL:** Constructs temporal subgraphs using a breadth-first search. It applies a graph transformer that integrates graph topology and temporal attributes while modeling interdependencies through cross-attention mechanisms.
- **GraphMixer:** Introduces fixed time encoding and uses an MLP-Mixer architecture for link and node features to create

Table 3: Statistics of datasets. Type: D = DTDG, C = CTDG; N.&F. Feat. = Node & Link Feature dimension. Graphs are classified as DTDGs if the number of links significantly exceeded the number of unique timesteps, as interactions within each batch primarily occurred within one single temporal snapshot.

Datasets	Type	Domains	Bipartite	#Nodes	#Links	#N.&L. Feat.	#Unique Steps	Duration	Time Granularity
Wikipedia	C	Social	True	9227	157474	- & 172	152757	1 month	Unix timestamps
Reddit	C	Social	True	10984	672447	- & 172	669065	1 month	Unix timestamps
MOOC	C	Interaction	True	7144	411749	- & 4	345600	17 months	Unix timestamps
LastFM	C	Interaction	True	1980	1293103	- & -	1283614	1 month	Unix timestamps
Myket	C	Interaction	True	17988	694121	- & -	693774	197 days	Unix timestamps
Enron	C	Social	False	184	125235	- & -	22632	3 years	Unix timestamps
UCI	C	Social	False	1899	59835	- & -	58911	196 days	Unix timestamps
Flights	D	Transport	False	13169	1927145	- & 1	122	4 months	days
Can. Parl.	D	Politics	False	734	74478	- & 1	14	14 years	years
US Legis.	D	Politics	False	225	60396	- & 1	12	12 congresses	congresses
UN Trade	D	Economics	False	255	507497	- & 1	32	32 years	years
UN Vote	D	Politics	False	201	1035742	- & 1	72	72 years	years
Contact	C	Proximity	False	692	2426279	- & 1	8064	1 month	5 minutes

effective representations. This model emphasizes simplicity while ensuring robustness across temporal graph tasks.

- **DyGFormer**: Incorporates attention mechanisms to handle dynamic graph structures and temporal sequences. Its architecture includes multiple transformer layers and employs co-occurrence and aligned encoding to enhance node representations. The model supports varying sequence lengths and dynamic contexts, offering high adaptability.

A.4 Detailed Experimental Configurations and Hyperparameters

We adopt a 70%-15%-15% chronological train-validation-test split, with 10% of nodes designated as unobserved to evaluate inductive performance. The model comprises 2 GRN blocks with 2 attention heads, using an embedding dimension of 64 for both node and temporal representations. Training is performed using the Adam optimizer with binary cross-entropy loss, a learning rate of 10^{-4} , and a batch size of 200. The noise schedule parameter λ is computed as $1 - \exp(\text{linspace}(\ln 2^{-6}, \ln 2^{-10}, 2))$. Layer-wise dropout rates are set to $[0.1, 0.3, 0.2, 0.1, 0.1, 0.2, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1]$, with optimal hyperparameters selected via grid search. Each model is trained for 50 epochs with early stopping (patience = 10) and evaluated over five runs with different random seeds initialized from system timestamps. Detailed baseline configurations are available in our code repository. All experiments are conducted on Ubuntu with an NVIDIA GeForce RTX 4090 GPU (24GB) and an Intel Core i9-13900KF CPU.

A.5 Clarification on Inconsistencies in Experimental Implementation and Settings

Inconsistent Evaluation Protocol for Inductive Setting. We note that the DyGLib follows the same inductive evaluation protocol as TGAT³ and TGN⁴, where unseen nodes are included in the validation and test sets under the transductive setting. In contrast, our experiments follow the protocol proposed by CAWN⁵, ensuring that unseen nodes are strictly excluded from the transductive setup.

Such discrepancies, particularly in the inclusion of unseen nodes during transductive evaluation and the use of early stopping, introduce inconsistencies in the comparative results of both transductive and inductive evaluations.

B Additional Experimental Results

B.1 Edge-Level Prediction Performance

We report the results of the comparison experiments with baselines for the link prediction task, presenting ROC-AUC scores under the transductive setting in Table 4, and both AP and ROC-AUC scores under the inductive setting in Tables 5 and 6.

Table 4: AUC-ROC for transductive link prediction on dynamic graphs with random negative sampling strategy.

Datasets	JODIE	DyRep	TGAT	TGN	CAWN	EdgeBank	TCL	GraphMixer	DyGFormer	GRN
Wikipedia	91.48±3.34	90.60±0.59	93.36±0.35	97.25±0.07	98.05±0.03	90.78±0.00	94.45±0.10	95.68±0.05	98.68±0.03	97.86±0.40
Reddit	97.56±0.16	97.42±0.12	96.58±0.04	96.93±0.16	98.63±0.03	95.20±0.00	93.21±0.17	95.26±0.04	98.86±0.06	99.47±0.03
MOOC	81.22±4.43	74.20±1.09	78.97±1.69	76.45±1.29	70.29±0.81	66.86±0.00	76.02±1.85	75.23±0.27	72.39±1.03	92.89±1.06
LastFM	69.91±0.66	69.79±1.41	63.03±0.33	53.33±1.75	81.44±0.10	83.77±0.00	56.36±0.19	68.28±0.02	90.10±0.04	72.47±0.62
Myket	86.37±0.12	86.35±0.13	66.04±5.60	82.69±2.08	83.72±0.16	57.35±0.00	68.81±2.18	85.00±0.04	78.99±0.39	82.36±1.94
Enron	83.27±0.65	75.67±1.41	65.10±0.53	75.35±2.50	85.44±0.90	87.09±0.00	67.52±1.53	84.12±0.14	86.51±2.44	84.50±2.87
UCI	83.66±1.13	56.73±4.00	76.08±0.68	85.87±0.94	91.78±0.23	77.30±0.00	84.37±0.78	89.77±0.75	93.94±0.11	80.74±1.54
Flights	93.59±0.42	91.49±0.83	91.63±0.02	93.28±1.55	96.57±0.29	90.23±0.00	90.45±0.03	90.44±0.00	98.74±0.04	96.04±1.33
Can. Parl.	82.76±0.52	75.24±1.14	72.31±1.27	75.47±1.57	72.62±3.97	64.14±0.00	69.59±3.44	81.66±1.11	94.93±1.28	90.47±0.13
US Legis.	85.62±0.14	75.42±2.36	64.66±3.96	77.92±0.27	70.36±2.76	62.57±0.00	60.75±0.99	77.77±0.56	78.94±0.27	86.38±0.32
UN Trade	71.84±0.48	69.03±1.09	62.87±0.44	65.47±0.26	65.32±0.11	66.75±0.00	63.10±0.17	54.17±0.33	60.00±2.14	72.33±0.52
UN Vote	68.72±1.60	65.45±0.88	52.78±0.62	61.12±1.43	52.33±0.06	62.97±0.00	50.88±0.20	52.37±0.34	54.50±0.12	86.01±0.97
Contact	95.00±0.15	88.53±1.56	95.11±0.07	94.18±1.41	85.31±0.19	94.34±0.00	92.27±0.22	92.25±0.24	98.15±0.05	91.02±0.59
Avg. Rank	3.69	6.23	7.08	5.38	5.15	6.92	7.85	6.15	3.31	3.23

B.2 Node-Level Classification Performance

The datasets chosen for the node classification task are Wikipedia, Reddit and MOOC. Wikipedia has 5 classes of node states, Reddit has 15 classes of node states and MOOC has 87 classes of node states. All three datasets contain nodes without original node features. It is worth noting that the distribution of node classes in these three datasets is relatively imbalanced. We report the experimental results for the node classification task in Table 7. The results of the

³<https://github.com/StatsDLMathsRecomSys/Inductive-representation-learning-on-temporal-graphs>

⁴<https://github.com/twitter-research/tgn>

⁵<https://github.com/snap-stanford/CAW>

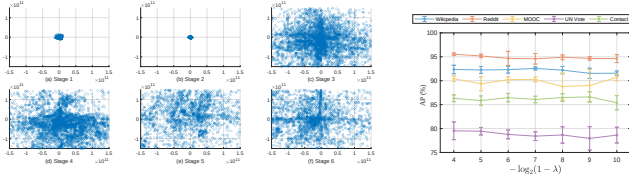


Figure 7: T-SNE visualization of recurrent states with $\lambda = 1 - 2^{-10}$. Each stage represents the same 5% interval of the input data within a training epoch.

Figure 8: Impact of decay strength with 95% CI on AP across five datasets under inductive setting.

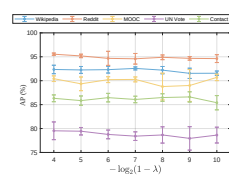


Table 5: AP for inductive link prediction on dynamic graphs with random negative sampling strategy.

Datasets	JODIE	DyRep	TGAT	TGN	CAWN	TCL	GraphMixer	DyGFormer	GRN
Wikipedia	86.38±4.17	84.29±0.88	93.91±0.24	96.95±0.12	97.97±0.01	95.17±0.11	95.41±0.09	98.35±0.07	95.69±0.57
Reddit	91.95±0.45	88.36±0.42	93.70±0.15	90.50±1.60	98.16±0.03	88.66±0.12	92.46±0.04	<u>98.46±0.04</u>	98.57±0.16
MOOC	70.50±2.51	64.37±2.06	<u>74.70±2.23</u>	74.16±3.33	64.71±0.97	71.14±1.51	68.35±0.65	64.39±0.94	90.59±0.77
LastFM	82.65±1.02	82.36±1.93	75.17±0.20	65.89±6.42	<u>87.55±0.09</u>	71.85±0.20	81.63±0.20	93.11±0.03	83.44±0.95
Myket	80.58±0.38	76.84±0.94	60.99±4.85	61.30±5.02	77.76±0.24	63.27±2.72	<u>80.50±0.03</u>	72.90±0.57	73.23±4.91
Enron	75.38±0.87	62.02±4.84	66.30±0.67	68.25±1.70	<u>82.58±0.57</u>	67.73±2.17	74.44±0.48	83.96±2.06	81.39±2.94
UCI	73.94±1.78	49.55±1.87	79.70±0.60	81.62±0.68	89.80±0.20	82.00±1.05	<u>90.16±0.47</u>	93.78±0.07	75.27±1.35
Flights	86.61±1.82	82.88±1.20	83.64±0.12	87.47±2.45	<u>94.56±0.35</u>	79.96±0.47	79.79±0.18	97.43±0.05	87.55±3.76
Can. Parl.	51.84±1.17	50.58±1.18	53.35±0.41	53.42±0.37	55.13±1.29	53.48±0.88	57.39±0.35	81.86±0.69	77.58±1.90
US Legis.	52.31±0.32	55.75±1.06	55.77±4.45	<u>59.69±0.15</u>	59.52±0.73	59.22±0.40	53.89±1.47	55.49±0.54	77.52±4.37
UN Trade	58.55±0.56	56.44±0.50	59.72±0.93	52.70±0.71	63.29±0.16	61.12±0.66	54.10±4.47	55.72±1.74	64.26±2.15
UN Vote	50.50±1.30	51.08±1.55	50.79±0.43	49.91±1.54	49.09±0.85	52.72±1.08	<u>54.43±0.43</u>	53.64±0.21	78.07±1.38
Contact	<u>93.14±0.63</u>	76.31±9.60	92.68±0.22	87.04±3.75	86.36±0.31	87.38±0.56	87.46±0.31	97.71±0.04	91.18±1.12
Avg. Rank	5.54	7.38	5.69	5.92	3.85	5.69	5.08	<u>3.08</u>	2.77

Table 6: AUC-ROC for inductive link prediction on dynamic graphs with random negative sampling strategy.

Datasets	JODIE	DyRep	TGAT	TGN	CAWN	TCL	GraphMixer	DyGFormer	GRN
Wikipedia	85.97±3.36	82.93±0.69	93.44±0.28	96.69±0.14	97.59±0.02	94.62±0.07	95.24±0.08	98.19±0.06	95.27±0.65
Reddit	91.43±0.43	87.81±0.56	93.85±0.15	91.40±1.19	97.78±0.04	89.11±0.12	92.39±0.03	<u>98.21±0.07</u>	98.90±0.17
MOOC	75.66±2.78	67.86±1.77	76.05±2.85	<u>76.07±3.82</u>	65.62±0.92	71.98±2.29	70.00±0.36	65.56±1.02	91.82±0.69
LastFM	82.12±1.09	81.64±1.98	74.67±0.18	64.02±5.23	<u>85.07±0.14</u>	69.83±0.18	80.35±0.05	92.87±0.05	82.11±1.04
Myket	77.71±0.45	74.48±1.22	60.37±5.60	59.88±6.34	74.54±0.22	60.91±2.46	<u>77.67±0.06</u>	69.57±0.53	72.60±5.53
Enron	76.53±1.94	62.37±6.40	64.13±0.96	67.50±1.46	<u>82.16±0.55</u>	65.46±2.49	75.70±0.60	83.12±2.57	78.95±3.15
UCI	75.25±1.09	40.14±2.81	76.97±0.77	78.27±0.70	87.13±0.24	79.72±0.92	<u>88.33±0.54</u>	91.54±0.11	72.37±1.36
Flights	87.16±1.40	82.28±1.42	83.67±0.08	87.98±3.11	<u>93.60±0.47</u>	80.49±0.10	80.38±0.05	97.35±0.09	88.59±4.46
Can. Parl.	49.44±1.31	47.29±2.04	54.01±0.73	55.44±0.79	58.22±2.07	55.19±1.50	58.79±0.49	84.95±1.06	79.15±1.14
US Legis.	55.10±0.33	58.01±1.72	56.27±5.33	<u>60.25±0.34</u>	59.40±1.39	59.72±0.47	54.46±2.65	63.46±0.65	81.96±2.93
UN Trade	60.39±0.57	58.11±0.53	61.64±0.52	54.65±1.01	65.05±0.18	62.69±0.41	53.77±6.18	57.60±2.40	<u>63.33±2.46</u>
UN Vote	47.75±2.08	49.91±2.26	50.45±0.48	48.77±1.91	47.43±0.57	51.54±1.15	<u>53.55±0.38</u>	53.23±0.24	80.54±0.86
Contact	94.20±0.32	79.92±6.85	<u>94.43±0.09</u>	90.95±2.42	84.30±0.30	90.93±0.23	90.93±0.24	97.96±0.04	91.10±0.75
Avg. Rank	5.31	7.23	5.62	5.54	3.92	5.77	5.31	<u>3.31</u>	3.00

comparison experiments for the node classification task are shown in Table 7.

Table 7: AUC-ROC for node classification on dynamic graphs.

Datasets	JODIE	DyRep	TGAT	TGN	CAWN	TCL	GraphMixer	DyGFormer	GRN
Wikipedia	<u>86.89±1.31</u>	87.72±0.71	68.93±3.06	75.82±2.60	79.17±1.07	75.85±2.18	72.38±2.89	78.99±0.12	88.32±1.45
Reddit	64.82±1.77	<u>62.25±1.50</u>	61.07±1.32	57.11±1.46	59.84±0.07	61.44±1.11	60.53±0.59	61.85±0.57	61.10±0.72
MOOC	69.04±1.27	<u>64.88±2.88</u>	59.92±0.72	70.26±1.09	65.24±1.06	67.89±0.79	64.19±0.59	<u>73.90±0.55</u>	76.79±2.20
Avg. Rank	<u>2.67</u>	3.67	8.00	6.33	6.00	5.00	7.67	3.33	2.33

B.3 Additional Loss Landscape Visualizations

To evaluate the generalization capability of GRNs, we visualize and compare the loss landscapes of GRNs and baseline models. Specifically, we give 2D filled contour plots for the Wikipedia, Reddit,

and MOOC datasets in our anonymous repository. These visualizations help assess the sharpness and smoothness of the optimization landscape, which are indicative of generalization behavior.

B.4 Additional Experimental Results of Recurrent State

We provide the additional results that further elucidate the contribution of the recurrent state and decay factor in Figures 7 and 8.

B.5 Additional Analysis of Ablation Study

In the transductive setting, the removal of the recurrent state S leads to a substantial drop in both AP and ROC-AUC, especially on Wikipedia. This underscores the critical role of temporal memory in capturing evolving node-level dynamics over continuous interactions. Similarly, eliminating the decay mechanism results in noticeable performance degradation. This confirms that regulating the temporal influence of past interactions via decay is essential for learning effective temporal representations, particularly in CT-DGs like Wikipedia. Interestingly, the absence of multi-scale decay only causes a marginal decline in performance, suggesting that while adaptive decay enhances robustness, a single-scale decay still retains reasonable modeling capacity. In contrast, removing temporal encoding causes the most severe performance deterioration on Wikipedia (dropping AP to nearly 80%) which highlights its necessity for distinguishing fine-grained timestamp patterns in high-resolution temporal graphs.

In the inductive setting, performance patterns remain consistent. The removal of temporal encoding again has the most pronounced negative effect on Wikipedia. This suggests that in inductive scenarios where generalization to unseen nodes is required and temporal encoding plays an even greater role in preserving event chronology. The impact of removing S and decay mechanisms is also evident, though somewhat less dramatic than in the transductive setting, especially on Reddit. This may be due to Reddit’s lower temporal granularity and more regular interaction patterns.

C Concerns and Limitations

The proposed GRN architecture demonstrated significant advancements, but several limitations warrant further investigation:

The current design of GRNs focuses on aggregating information from neighboring nodes, limiting its ability to capture high-order dependencies. This limitation may hinder the model’s performance in scenarios requiring a deeper understanding of graph structures or relationships spanning multiple hops.

Although the GRN effectively retains substantial information in its recurrent state, the fixed dimensionality of this state presents a scalability challenge. For super-large-scale and ultra-long-term dynamic graphs, increasing the dimensionality may be necessary to alleviate over-squashing. However, such modifications, could introduce significant computational overhead, necessitating careful tradeoffs between efficiency and expressiveness.